

PATCH 5

Correções, Cenário e Câmara Orbital

Mini-Jogo de Basquetebol 3D — ICG 2025/2026

Este patch corrige bugs críticos de câmara e de geometria do cesto, ajusta o cenário com NPCs revistos e bancos mais realistas, e adiciona controlo orbital da câmara em 3ª pessoa com rato e touchpad.

Índice do Patch 5

#	Melhoria	Ficheiro(s)	Dificuldade
5.1	Fix câmara 1ª pessoa — aro no lado oposto	core/camera.js, main.js	Baixa
5.2	Fix aro dentro da tabela	core/constants.js	Baixa
5.3	NPCs revistos — criança, parada e 2 a passear	world/npcs.js	Média
5.4	Bancos mais realistas	world/park.js	Média
5.5	Câmara orbital semi-livre (rato/touchpad)	core/camera.js, main.js	Média

5.1 — Fix Câmara 1ª Pessoa: Aro no Lado Oposto

Em 1ª pessoa ao iniciar um lançamento, o lock-on vira sempre o jogador para o cesto do lado oposto ao correto, independentemente da posição no campo. Este comportamento confirma que o vetor de direção para o cesto está invertido — está a calcular $headPos - hoop.rimCenter$ em vez de $hoop.rimCenter - headPos$.

Diagnóstico:

Sintoma	Causa
Vira sempre para o lado oposto	Vetor toHoop invertido — aponta para trás em vez de para o cesto
Acontece em qualquer posição do campo	Erro é sistemático, não depende de posição — é sempre sinal trocado
Não está relacionado com findNearHoop	O lock-on correto é encontrado mas a direção calculada é a oposta

Código com o bug e correção em camera.js:

Errado (vetor invertido):

```
// ERRADO — aponta DO cesto PARA a cabeça (sentido oposto) const toHoop = new THREE.Vector3(
headPos.x - hoop.rimCenter.x, 0, headPos.z - hoop.rimCenter.z );
```

Correto:

```
// CORRETO — aponta DA cabeça PARA o cesto const toHoop = new THREE.Vector3(
hoop.rimCenter.x - headPos.x, 0, hoop.rimCenter.z - headPos.z ); // Angulo horizontal e
vertical para o cesto: const targetYaw = Math.atan2(toHoop.x, toHoop.z); const targetPitch
= -Math.atan2( hoop.rimCenter.y - headPos.y, toHoop.length() ); // negativo = olhar para
cima // Aplicar por lerp suave (~0.35s de lock-on): fpYaw = lerpAngle(fpYaw, targetYaw, 1 -
Math.exp(-delta * 12)); fpPitch = lerpAngle(fpPitch, targetPitch, 1 - Math.exp(-delta *
12));
```

Durante o lock-on e o voo da bola o rato fica bloqueado. Só é desbloqueado quando a bola pousa ou entra no cesto.

5.2 — Fix Aro Dentro da Tabela

O aro está visualmente dentro da tabela porque o offset entre a tabela e o aro é demasiado pequeno. Com o aro maior do Patch 1 (rimRadius 0.34), parte do aro sobrepõe-se com a geometria da tabela, impedindo a bola de entrar e criando colisões falsas com o corpo da tabela.

Análise das constantes atuais:

Constante	Valor Atual	Problema
hoopOffsetFromBackboard	0.15 m	Distancia da face da tabela ao centro do aro — demasiado pequeno
backboardDepth	0.05 m	Espessura da tabela — o aro começa a 0.15m mas a tabela ocupa
rimRadius	0.34 m (Patch 1)	Com este raio o aro chega virtualmente ate a tabela

Valores corrigidos em constants.js:

Constante	Atual	Novo	Razao
hoopOffsetFromBackboard	0.15	0.40	Afasta o aro da tabela — fica claramente fora e visivel
backboardDepth	0.05	0.05	Mantem-se — a tabela e fina, o problema era o offset

Com `hoopOffsetFromBackboard = 0.40`, o centro do aro fica a 40cm da face da tabela, garantindo que mesmo com `rimRadius = 0.34` o aro nao toca nem se sobrepoe a tabela. A constante `getRimCenter()` em `constants.js` usa este valor diretamente, por isso a fisica e a detetacao de cesto atualizam automaticamente.

5.3 — NPCs Revistos: Crianca, Pessoa Parada e 2 a Passear

O conjunto de NPCs foi simplificado — removidos a pessoa sentada a ler e o casal em loop. Ficam 4 figuras distribuidas pelo parque com comportamentos distintos e animacoes procedurais suaves.

NPC	Escala	Comportamento	Animacao	Posicao
Crianca	0.55×	Loop oval pequeno, velocidade 0.2	Pernas balançam rapido (freq=1.5)	Então o NPC anda para a esquerda
Pessoa parada	1.0×	Estatica no lugar	Torso oscila levemente com $\sin(\text{time} \times 0.5)$	Pessoa com o corpo inclina li
Pessoa a passear	1.0×	Path linear lento, vel. 1.0	Pessoa balota alternadamente para a direita e esquerda	Faz dois passos para a direita
Pessoa a passear	1.05×	Path diagonal lento, vel. 0.8	Pessoa faz uma volta fase diferente dos outros	Anda para cima e para a direita

Sistema de paths em npcs.js:

```
// Cada NPC com path linear usa um progresso 0→1→0 (ida e volta) function
updateWalker(npc, delta) { npc.progress += delta * npc.speed * npc.direction; if
(npc.progress >= 1) { npc.progress = 1; npc.direction = -1; } if (npc.progress <= 0) {
npc.progress = 0; npc.direction = 1; } // Interpolar entre start e end da path
npc.group.position.lerpVectors( npc.pathStart, npc.pathEnd, npc.progress ); // Virar o NPC
na direcao do movimento const dir = npc.direction > 0 ? 1 : -1; const targetYaw =
Math.atan2( (npc.pathEnd.x - npc.pathStart.x) * dir, (npc.pathEnd.z - npc.pathStart.z) *
dir ); npc.group.rotation.y = lerpAngle( npc.group.rotation.y, targetYaw, 1 -
Math.exp(-delta * 5) ); // Animacao de andar npc.animTime += delta; const swing =
Math.sin(npc.animTime * 5) * 0.4; npc.leftLeg.rotation.x = swing; npc.rightLeg.rotation.x =
-swing; npc.leftArm.rotation.x = -swing * 0.5; npc.rightArm.rotation.x = swing * 0.5; }
```

5.4 — Bancos Mais Realistas

Os bancos atuais sao um BoxGeometry simples. Sero substituidos por bancos com assento de ripas de madeira, costas com travessas verticais e pernas metalicas — sem ficheiros externos, apenas geometrias Three.js compostas.

Parte	Geometria	Material	Cor / Detalhe
Ripa do assento (x3)	BoxGeometry(1.8, 0.06, 0.14)	MeshStandardMaterial	Castanho escuro #8B5E3C, espacadas 0.1
Travessa das costas (x3)	BoxGeometry(1.8, 0.06, 0.10)	MeshStandardMaterial	Mesmo castanho, inclinadas 15° para tras (ro
Suporte lateral (x2)	BoxGeometry(0.06, 0.72, 0.48)	MeshStandardMaterial	Castanho escuro #5D3A1A, une assento e co
Perna frontal (x2)	CylinderGeometry(0.025, 0.025, 0.44, 8)	MeshStandardMaterial	Cinza metálico #78909C, 0.3
Perna traseira (x2)	CylinderGeometry(0.025, 0.025, 0.44, 8)	MeshStandardMaterial	Qualquer cor, 0.7 no roughness para dar estabili
Travessa do chao (x2)	CylinderGeometry(0.018, 0.018, 0.52, 8)	MeshStandardMaterial	Ligadas as pernas traseiras horizontalm

Organizacao em createBench(x, z, rotY) em park.js:

```
// Cada banco e um Group com todos os elementos
export function createBench(scene, x, z, rotY = 0) {
  const group = new THREE.Group();
  const woodMat = new THREE.MeshStandardMaterial({
    color: 0x8B5E3C, roughness: 0.85, metalness: 0.0 });
  const metalMat = new THREE.MeshStandardMaterial({
    color: 0x78909C, roughness: 0.30, metalness: 0.7 });
  // 3 ripas do assento [-0.16, 0, 0.16].forEach((offset, i) => {
  const ripa = new THREE.Mesh( new THREE.BoxGeometry(1.8, 0.06, 0.14), woodMat);
  ripa.position.set(0, 0.44, offset);
  ripa.castShadow = true;
  group.add(ripa);
  });
  // 3 travessas das costas (inclinadas) [-0.10, 0, 0.10].forEach(offset => {
  const travessa = new THREE.Mesh( new THREE.BoxGeometry(1.8, 0.06, 0.10), woodMat);
  travessa.position.set(0, 0.68, offset - 0.18);
  travessa.rotation.x = -0.26; // inclinada 15 graus
  travessa.castShadow = true;
  group.add(travessa);
  });
  // Pernas metalicas (frontal e traseira em cada lado)
  [[-0.82, 0.10], [0.82, 0.10], [-0.82, -0.20], [0.82, -0.20]].forEach(([px, pz]) => {
  const leg = new THREE.Mesh( new THREE.CylinderGeometry(0.025, 0.025, 0.44, 8), metalMat);
  leg.position.set(px, 0.22, pz);
  leg.castShadow = true;
  group.add(leg);
  });
  group.position.set(x, 0, z);
  group.rotation.y = rotY;
  scene.add(group);
  return group;
}
```

5.5 — Camera Orbital Semi-Livre (Rato / Touchpad)

Em 3a pessoa, mover o rato ou o touchpad horizontalmente roda a camara em torno do jogador. Apenas o eixo horizontal (yaw) e controlavel — a altura da camara mantem-se fixa para evitar angulos estranhos. O WASD continua a mover o jogador relativo ao angulo atual da camara.

Comportamento detalhado:

Aspeto	Detalhe
Input	mousemove event — movementX usado para rotacao horizontal. Funciona igual em rato e touchpad
Sensibilidade	cameraYaw -= e.movementX * 0.003 (valor ajustavel em constantes)
Eixo controlado	Apenas yaw (horizontal). Pitch (vertical) mantem-se fixo no offset definido.
Pointer Lock	NAO e usado em 3a pessoa — o cursor mantem-se visivel para o HUD e botoes.
Movimento WASD	Passa a ser relativo ao cameraYaw em vez de um yaw fixo — jogador move-se na direcao da camara
Camara ao cruzar meio campo	On flip automatico de 180 graus do Patch 2 mantem-se mas e suavizado pelo mesmo lerp
Reset da camara	Tecla C reseta o cameraYaw para atras do jogador (playerYaw + pi).

Implementacao em camera.js:

```
// Estado da camara orbital let cameraYaw = 0; // angulo horizontal controlado pelo rato let
cameraYawTarget = 0; // Handler de mousemove – adicionar em core/camera.js ou main.js
window.addEventListener('mousemove', (e) => { if (isFirstPerson) return; // em 1a pessoa
usa outro sistema cameraYaw -= e.movementX * CAM.orbitSensitivity; }); // Em update() –
calcular posicao da camara com o yaw orbital function update(playerPos, delta) { // Suavizar
o yaw cameraYaw = lerpAngle(cameraYaw, cameraYawTarget, 1 - Math.exp(-delta * 8)); //
Calcular offset rotacionado pelo cameraYaw const offset = new THREE.Vector3(CAM.offsetX,
CAM.offsetY, CAM.offsetZ); offset.applyAxisAngle(new THREE.Vector3(0,1,0), cameraYaw); //
Posicao desejada const desired = playerPos.clone().add(offset);
camera.position.lerp(desired, 1 - Math.exp(-delta * 8)); camera.position.y = Math.max(1.2,
camera.position.y); // Olhar para o jogador const lookAt = playerPos.clone(); lookAt.y +=
1.2; camera.lookAt(lookAt); }
```

Constantes a adicionar em constants.js:

```
export const CAM = { offsetX: 0, offsetY: 3.2, offsetZ: 5.5, orbitSensitivity: 0.003, //
ajustar ao gosto lerpSpeed: 8, };
```

Movimento WASD relativo a camara:

Com a camara orbital, o WASD deve mover o jogador na direcao que a camara esta a olhar, nao uma direcao fixa. Em updatePlayerMovement() em main.js:

```
// ANTES – direcao fixa (playerYaw fixo) tempVector.set(Math.sin(playerYaw), 0,
Math.cos(playerYaw)); // AGORA – direcao relativa ao angulo da camara const forward = new
THREE.Vector3( Math.sin(cameraYaw), 0, Math.cos(cameraYaw) ); const right = new
THREE.Vector3( Math.cos(cameraYaw), 0, -Math.sin(cameraYaw) ); const moveDir =
forward.multiplyScalar(moveZ) .add(right.multiplyScalar(moveX)) .normalize(); // 0
playerYaw (direcao do modelo) segue a direcao do movimento: if (moveDir.length() > 0.01) {
const targetYaw = Math.atan2(moveDir.x, moveDir.z); playerYaw = lerpAngle(playerYaw,
targetYaw, 1 - Math.exp(-delta * 10)); }
```

Nota: o cameraYaw começa como playerYaw + Math.PI (atrás do jogador) e o jogador ao mover-se não arrasta a camara — o utilizador controla a camara independentemente com o rato.

Ordem de Implementacao do Patch 5

Comecar pelos fixes criticos que afetam jogabilidade, depois o cenario e por fim a camara orbital.

Orde m	Melhoria	Razao
1o	5.2 — Fix aro dentro da tabela	Uma linha em constants.js — fix critico que impede golos
2o	5.1 — Fix camara 1a pessoa	Trocar sinal do vetor toHoop — fix critico de uma linha
3o	5.4 — Bancos mais realistas	Isolado em park.js, substituir geometria existente
4o	5.3 — NPCs revistos	Simplificar npcs.js — remover cenas e ajustar paths
5o	5.5 — Camara orbital	O mais complexo — afeta input, camera e movimento. Testar bem.

Ferramentas de IA integradas no desenvolvimento e apoio a implementacao: OpenAI ChatGPT · Anthropic Claude